Name: Shamrudha Varshini P **Roll No: 231901048**

Ex. No: 4 Querying Ledger Data with Hyperledger Fabric

Aim

To query and retrieve ledger data from a Hyperledger Fabric blockchain network using JavaScript chaincode, and verify asset information stored in the world state database.

Software Requirements

Kali Linux / Ubuntu
Docker
Docker Compose
Node.js (Version 18 or later)
npm
Git
Hyperledger Fabric Samples
Hyperledger Fabric Binaries
Hyperledger Fabric Docker Images Visual Studio Code (Optional)

Hardware Requirements

Processor: Intel Core i3 or above
RAM: Minimum 8 GB
Storage: Minimum 20 GB Free Disk Space Internet Connection

Procedure

Step 1: Navigate to the Fabric Test Network

Open the terminal and navigate to the Hyperledger Fabric test network.

```
cd ~/fabric-dev/fabric-samples/test-network
```

Expected Output:

```
(directory changed, no output)
```

Step 2: Start the Hyperledger Fabric Network

Start the Fabric network and create the application channel.

```
./network.sh up createChannel -ca
```

Expected Output:

```
Creating channel 'mychannel'
Starting peer0.org1
Starting peer0.org2
Starting orderer.example.com
```

```
Network Started Successfully
```

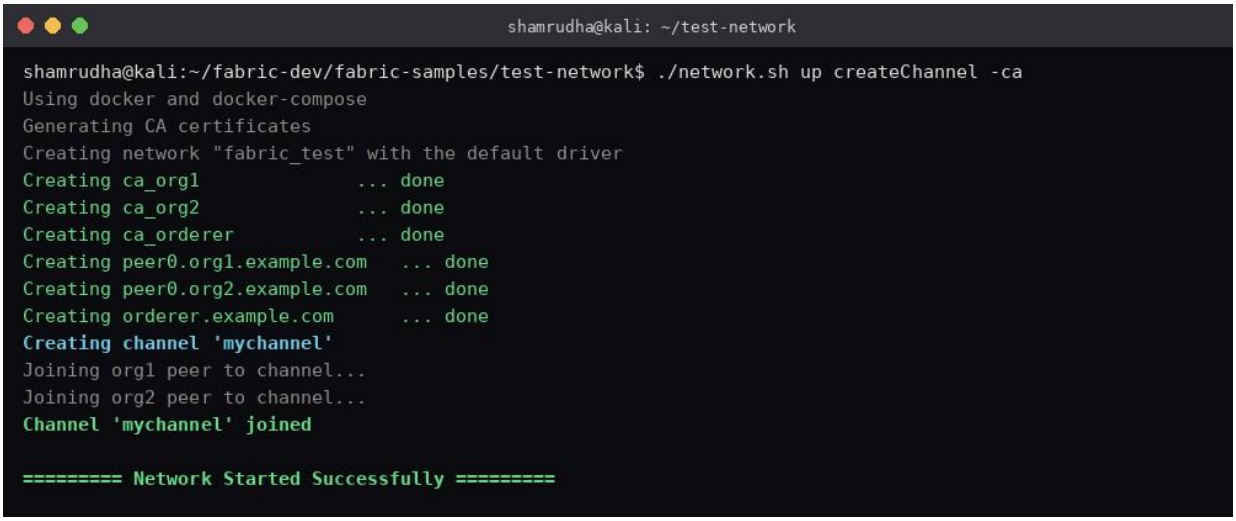


Fig 4.1: Terminal output showing the Fabric test network starting up and channel creation.

Step 3: Deploy the Asset Transfer Chaincode

Deploy the JavaScript Asset Transfer chaincode.

```
./network.sh deployCC \
-ccl javascript \
-ccn basic \
-ccp ../asset-transfer-basic/chaincode-javascript
```

Expected Output:

```
Packaging Chaincode
Installing Chaincode
Approving Chaincode
Committing Chaincode
Chaincode committed successfully
```

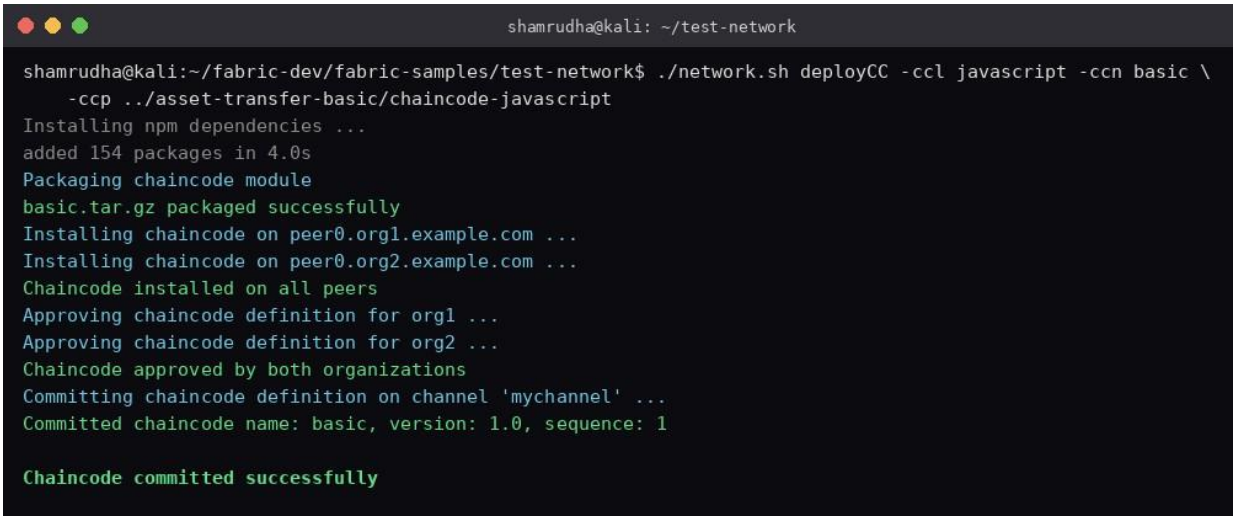


Fig 4.2: Chaincode packaging, installation, approval, and commit sequence.

Step 4: Initialize the Ledger

Load the ledger with sample assets.

```
peer chaincode invoke \ -
o localhost:7050 \
--ordererTLSHostnameOverride orderer.example.com \
--tls \
--cafile $ORDERER_CA \
-C mychannel \
-n basic \
-c '{"function":"InitLedger","Args":[]}'
```

Expected Output:

```
Ledger Initialized Successfully
```

Fig 4.3: InitLedger transaction invoked and committed successfully.

Step 5: Query All Assets

Retrieve all asset records stored in the blockchain ledger.

```
peer chaincode query \ -
C mychannel \
-n basic \
-c '{"Args":["GetAllAssets"]}'
```

Expected Output:

```
[
{"ID":"asset1","Color":"blue","Size":5,"Owner":"Tomoko","AppraisedValue":3
00},
{"ID":"asset2","Color":"red","Size":5,"Owner":"Brad","AppraisedValue":400}
]
```

Fig 4.4: Query result showing all assets stored on the ledger.

Step 6: Query a Specific Asset Retrieve

the details of asset1.

```
peer chaincode query \ -
C mychannel \
```

```
-n basic \
-c '{"Args":["ReadAsset","asset1"]}'
```

Expected Output:

```
{
"ID":"asset1",
"Color":"blue",
"Size":5,
"Owner":"Tomoko",
"AppraisedValue":300
}
```

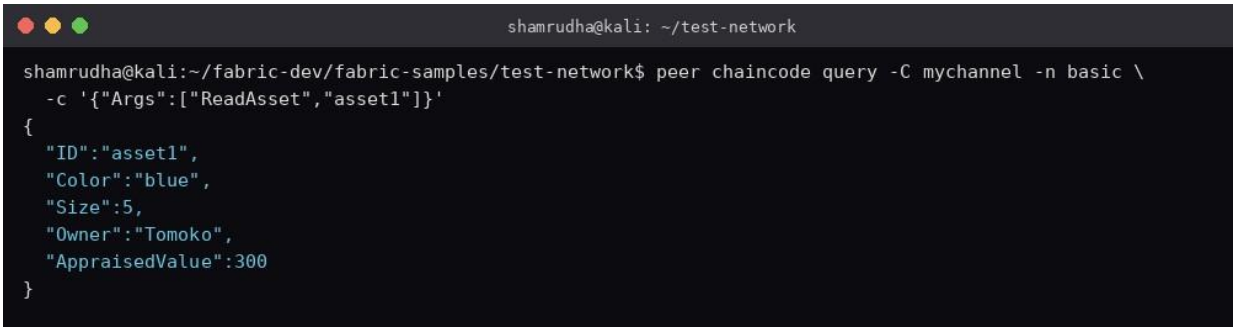


Fig 4.5: Query result showing the details of a single specific asset.

Step 7: Create a New Asset

Invoke the chaincode to create a new asset on the ledger.

```
peer chaincode invoke \ -
o localhost:7050 \
--ordererTLSHostnameOverride orderer.example.com \
--tls \
--cafile $ORDERER_CA \
-C mychannel \
-n basic \
-c
'{"function":"CreateAsset","Args":["asset20","Green","8","Arun","1500"]}'
```

Expected Output:

```
Asset asset20 created successfully
```

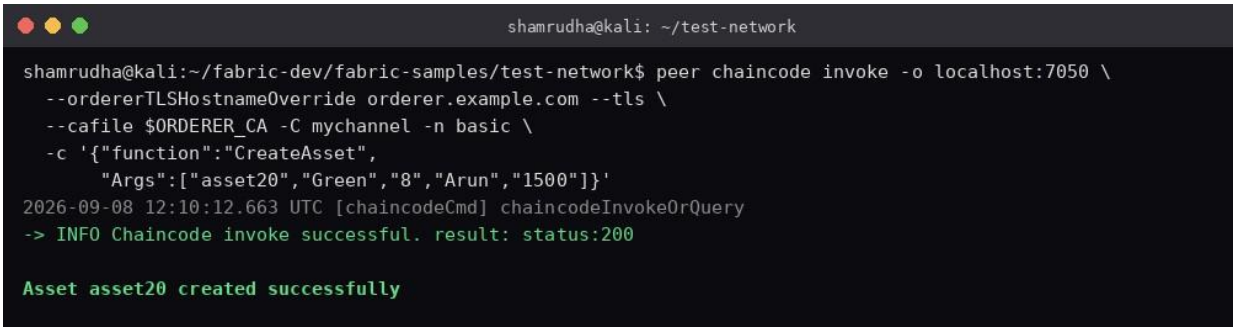


Fig 4.6: New asset 'asset20' created successfully on the ledger.

Step 8: Query the Newly Created Asset

Retrieve the details of the newly created asset.

```
peer chaincode query \ -
C mychannel \
```

```
-n basic \
-c '{"Args":["ReadAsset","asset20"]}'
```

Expected Output:

```
{
"ID":"asset20",
"Color":"Green",
"Size":8,
"Owner":"Arun",
"AppraisedValue":1500
}
```

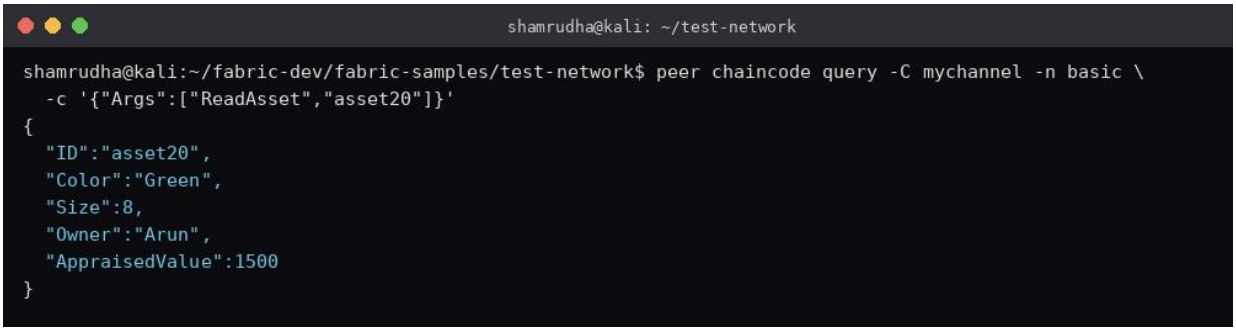


Fig 4.7: Ledger query confirming the details of the newly created asset.

Step 9: Query All Assets Again

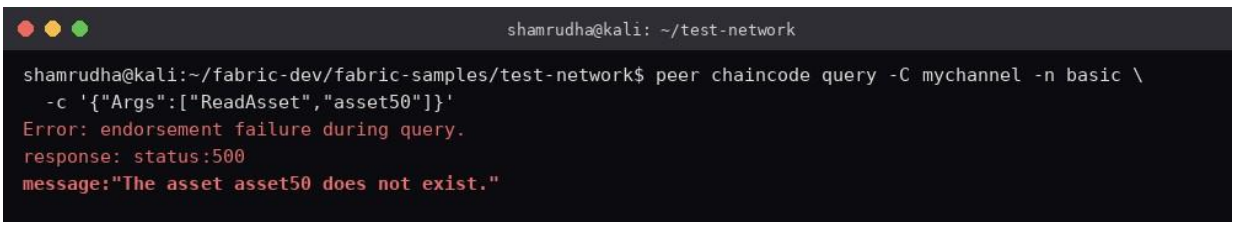
Retrieve all assets to verify that the new asset has been added.

```
peer chaincode query \ -
C mychannel \
-n basic \
-c '{"Args":["GetAllAssets"]}'
```

Expected Output:

```
List of all assets including asset20 displayed successfully.
```

Fig 4.8: Updated ledger listing showing asset20 alongside the original assets.



Step 10: Query a Non-Existing Asset

Attempt to retrieve an asset that does not exist.

```
peer chaincode query \ -
C mychannel \
-n basic \
-c '{"Args":["ReadAsset","asset50"]}'
```

Expected Output:

```
Error: The asset asset50 does not exist.
```

Fig 4.9: Query error confirming that a non-existing asset cannot be retrieved.

Step 11: Stop the Fabric Network

Bring down the Hyperledger Fabric test network and clean up the containers.

```
./network.sh down
```

Expected Output:

```
Network Stopped Successfully
```

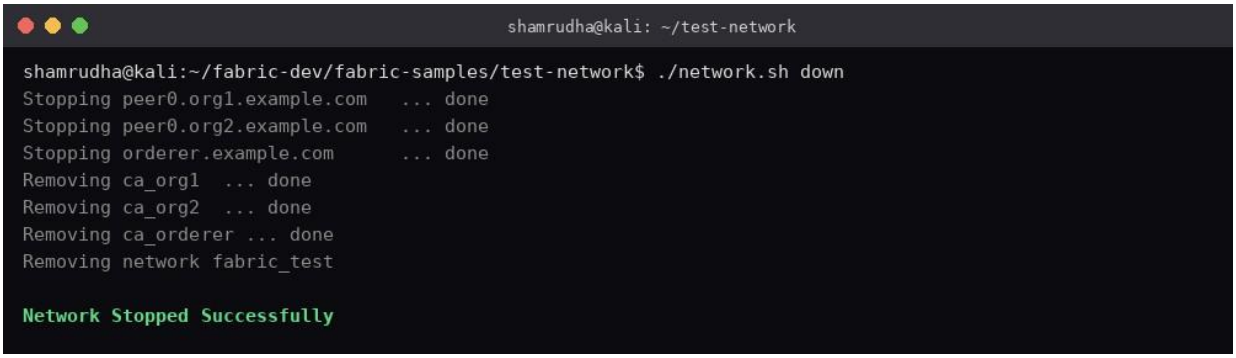
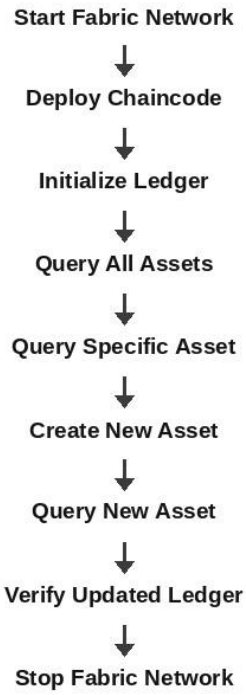


Fig 4.10: Fabric test network and associated Docker containers stopped.

Flow of Ledger Query Process



Result

The ledger data stored in the Hyperledger Fabric blockchain network was successfully queried using JavaScript chaincode. Asset records were retrieved from the world state database, specific asset details were verified, and newly created assets were successfully queried, demonstrating efficient data retrieval and verification in a permissioned blockchain environment.